

FROM CLOUD TO GPU: INSIDE THE MODERN VECTOR TILE RENDERING PIPELINE

Markus Tremmel
FOSS4G Hiroshima 2026

ROHDE & SCHWARZ

Make ideas real



The whole planet, navigable in a browser tab.

60 fps

interactive rendering

~100 GB

planet vector tiles

0

tile servers → served straight from
object storage

Cloud-optimized tiles + a GPU-driven renderer.



High-performance maps.



Industry

Staff Software Engineer & Architect at Rohde & Schwarz, focused on high-performance map & geospatial applications



Open Source

Creator of the MapLibre Tile (MLT) and COMTiles formats

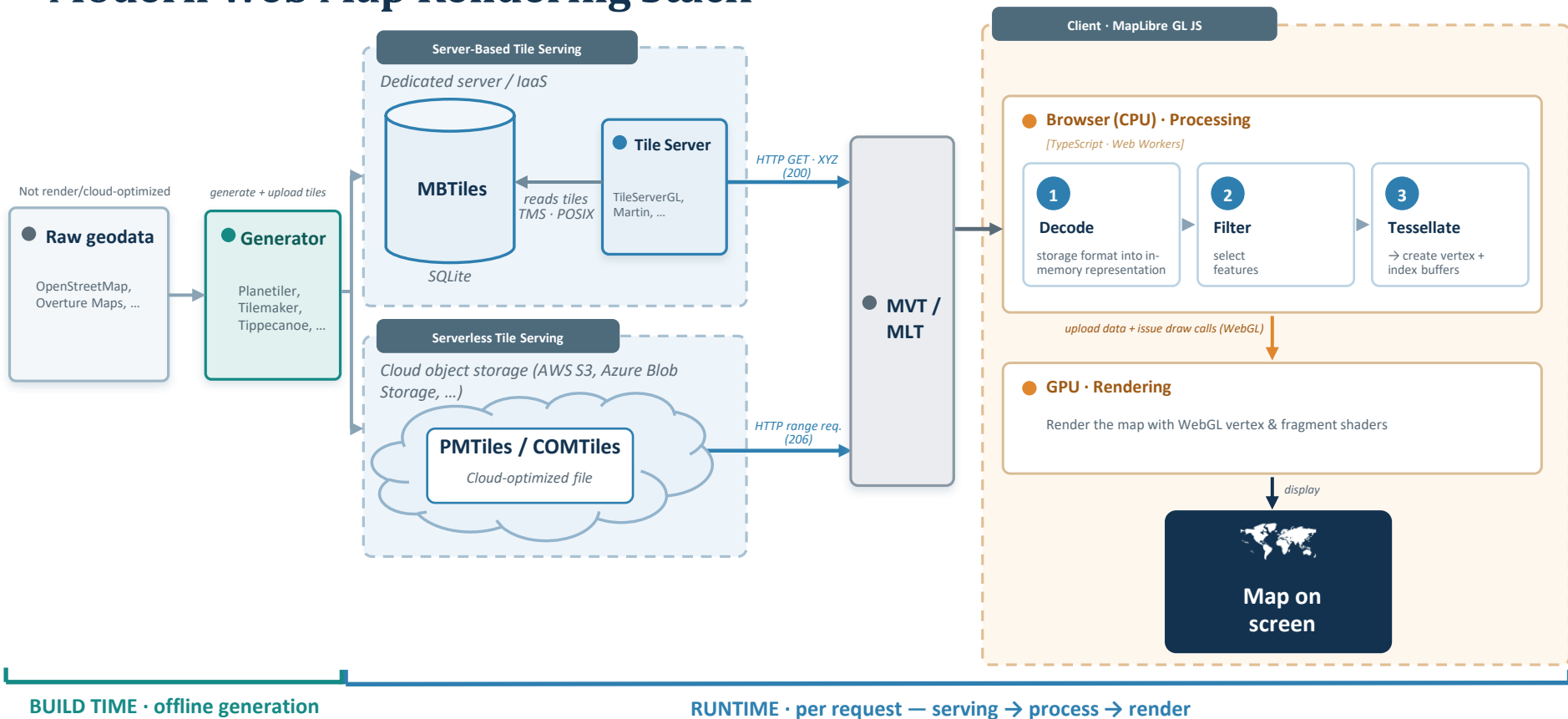


Academia

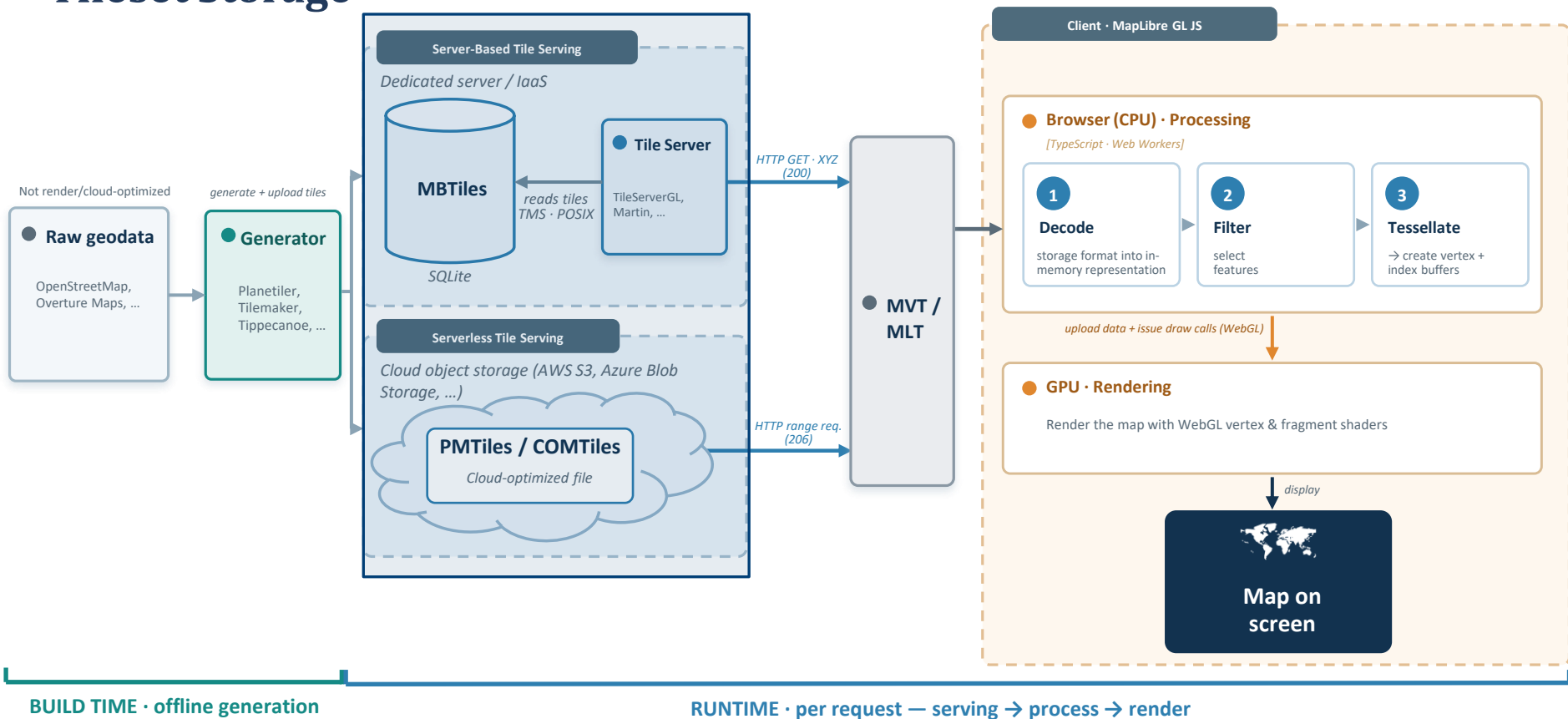
PhD research on the next-gen (cloud- & GPU-native) rendering stack · teaching vector data processing at Deggendorf Institute of Technology



Modern Web Map Rendering Stack



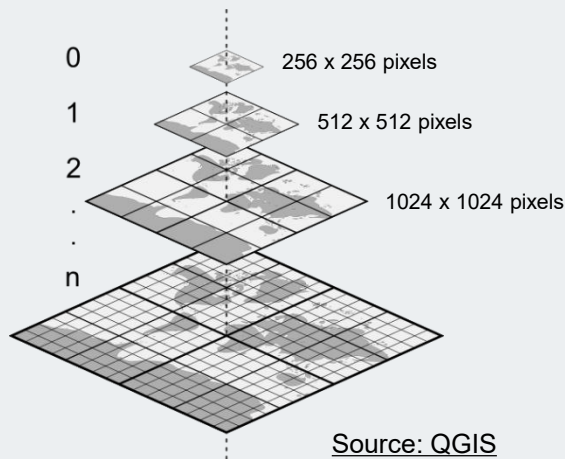
Tileset Storage



RASTER TILES

Pre-rendered images of the vector data, encoded as PNG, JPEG or WebP, ...

- **Less powerful hardware needed** → tiles are already pre-rendered
- **No GPU needed** → works on all type of devices and browsers
- **Trivial map client implementations**



VECTOR TILES

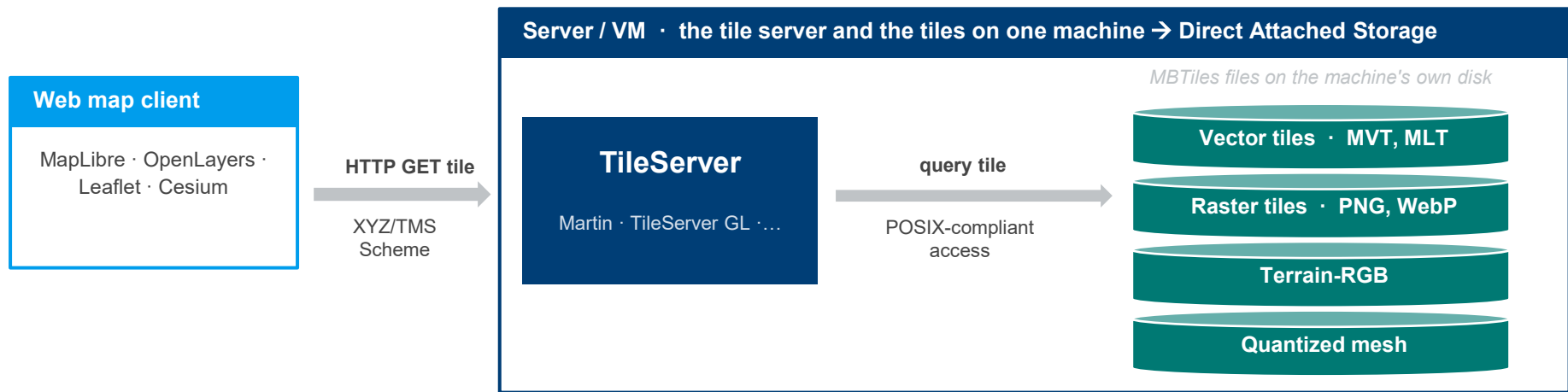
Contains the **vector data itself** (points, lines, polygons) rendered on the fly in the client, usually on the GPU, in the browser via WebGL

- **Dynamic styling** → The client decides on styling
- **Smooth, continuous zoom**
- **Interactivity**
- **Smaller tiles** → offline maps become practical
- Rotation of the map while **labels stay upright**



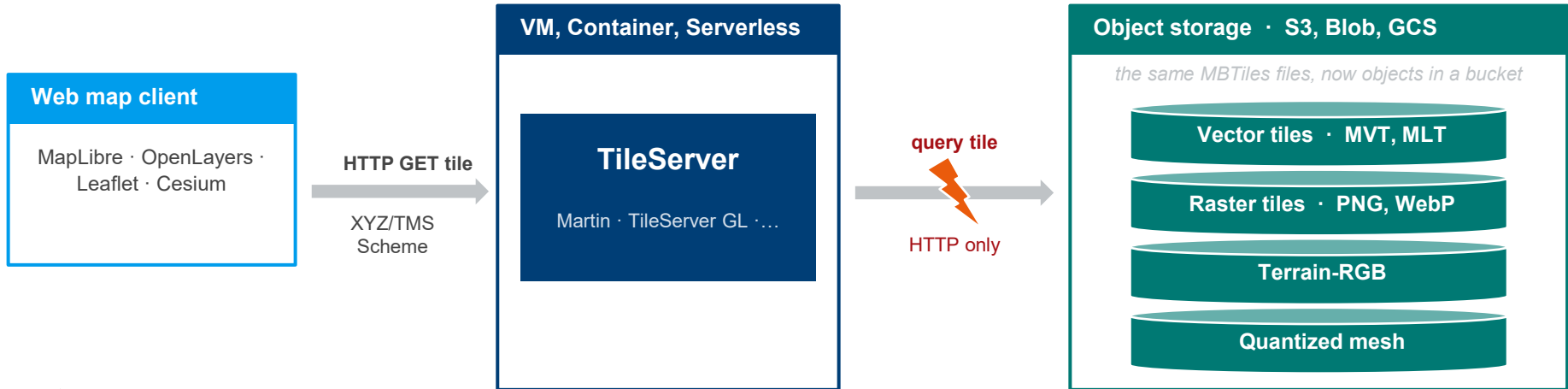
TRADITIONAL TILE ARCHIVES

- ▶ A planet-scale basemap up to zoom 15 is the sum of 4^z tiles $\approx$ 1.4 billion objects
- ▶ Classical tile archives, such as MBTiles and OGC GeoPackage, are based on a SQLite database and enable efficient storage and SQL-based querying of tiles



TRADITIONAL TILE ARCHIVES

- ▶ A planet-scale basemap up to zoom 15 is the sum of 4^z tiles $\approx$ 1.4 billion objects
- ▶ Classical tile archives, such as MBTiles and OGC GeoPackage, are based on a SQLite database and enable efficient storage and SQL-based querying of tiles
- ▶ In cloud architectures, however, a basic principle is the separation of storage and compute for scalability reasons

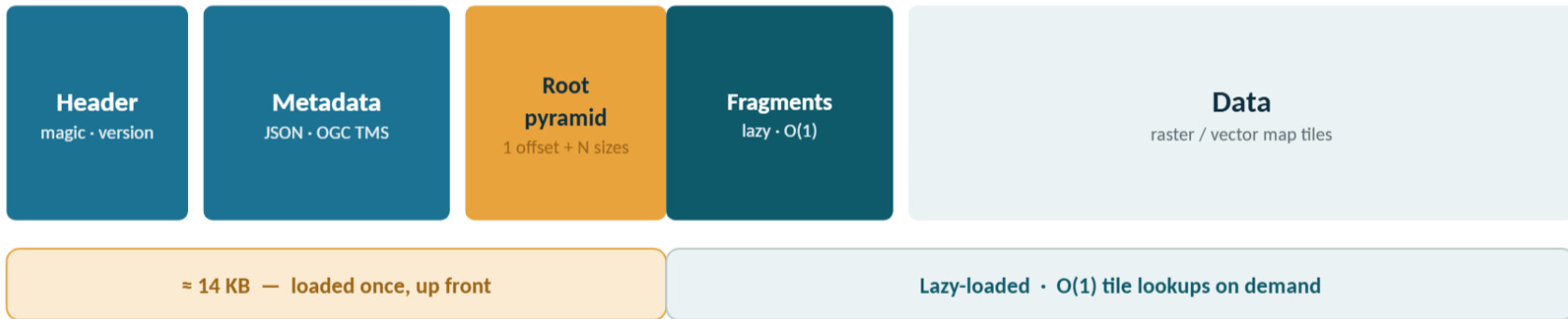


CLOUD OPTIMIZED TILE ARCHIVES

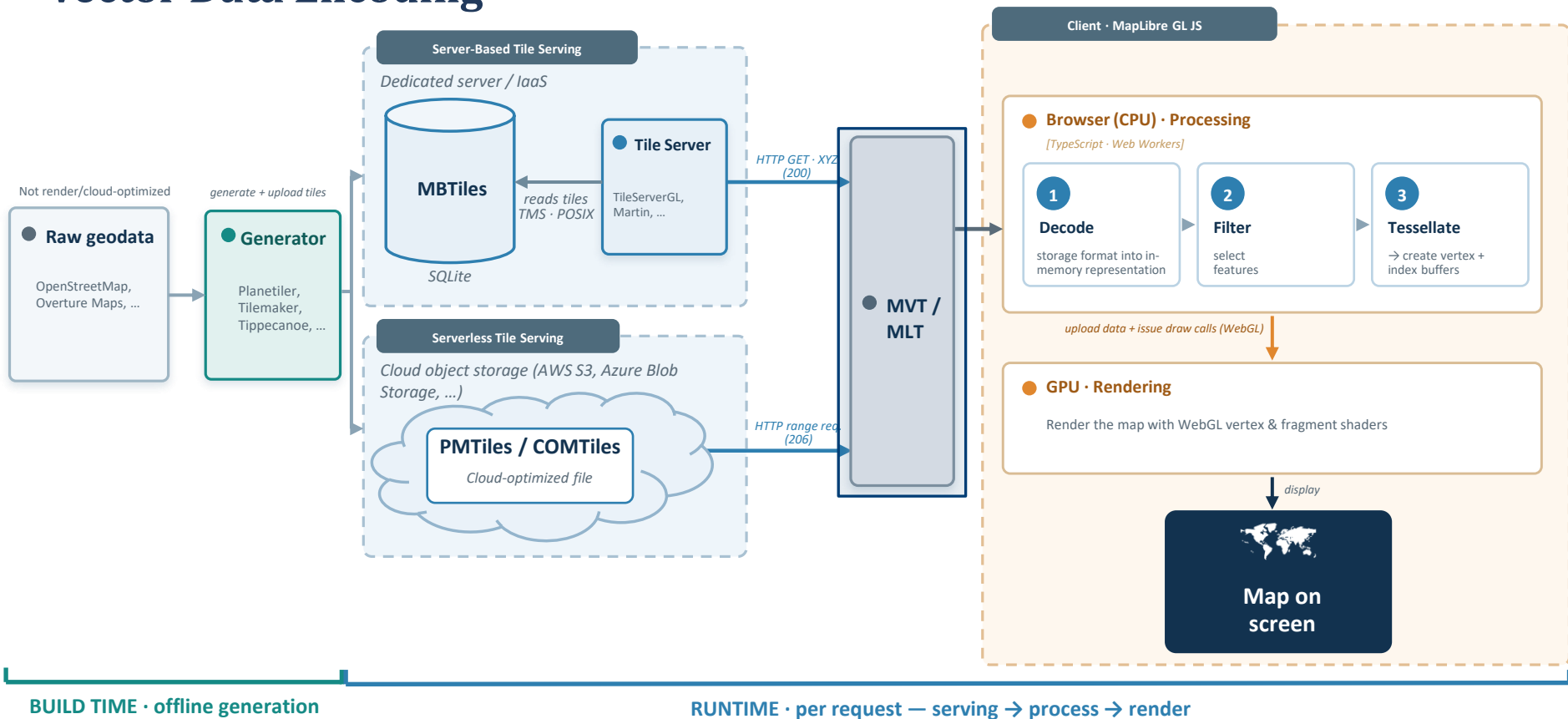
- ▶ Cloud-optimized formats support efficient access to selected portions of a file stored in cloud object storage, such as AWS S3, via HTTP Range requests
- ▶ PMTiles and COMTiles package large tile collections for cloud-native distribution without requiring a dedicated tile backend, enabling direct browser access to tiles stored in cloud object storage

COMTiles File Layout

INDEX — 2 TIERS

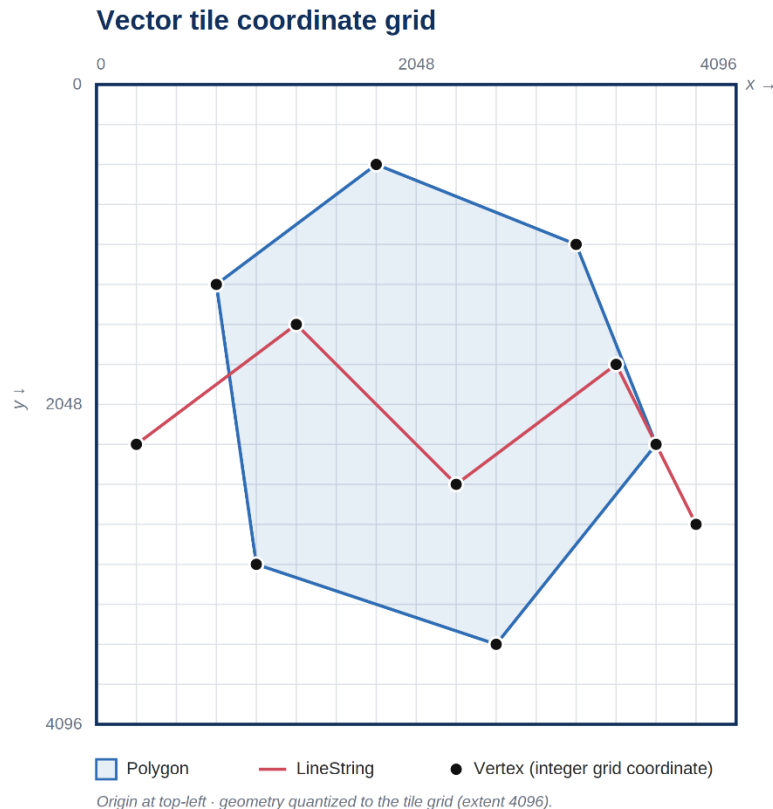


Vector Data Encoding



ANATOMY OF A VECTOR TILE

- ▶ Logical model: Tile → Layer / FeatureTable → Feature → { id, geometry, attributes }
- ▶ Geometry model is based on OGC Simple Feature Access standard: point, linestring, polygon and their multipart forms
- ▶ Coordinates are quantised to a tile-local integer grid (e.g., 4096×4096)
- ▶ A vector tile carries no styling information → appearance is defined by an external style document, e.g. the MapLibre Style Spec



VECTOR TILE STANDARDS

► Mapbox Vector Tile (MVT):

- the de facto industry standard for vector tile delivery, supporting up to 2.5D visualization
- stores feature data in a row-oriented layout
- relies heavily on general-purpose compression (gzip, Zstandard, ...)

2.5D rendering of LOD1 buildings using MapLibre GL JS



VECTOR TILE STANDARDS

► Mapbox Vector Tile (MVT):

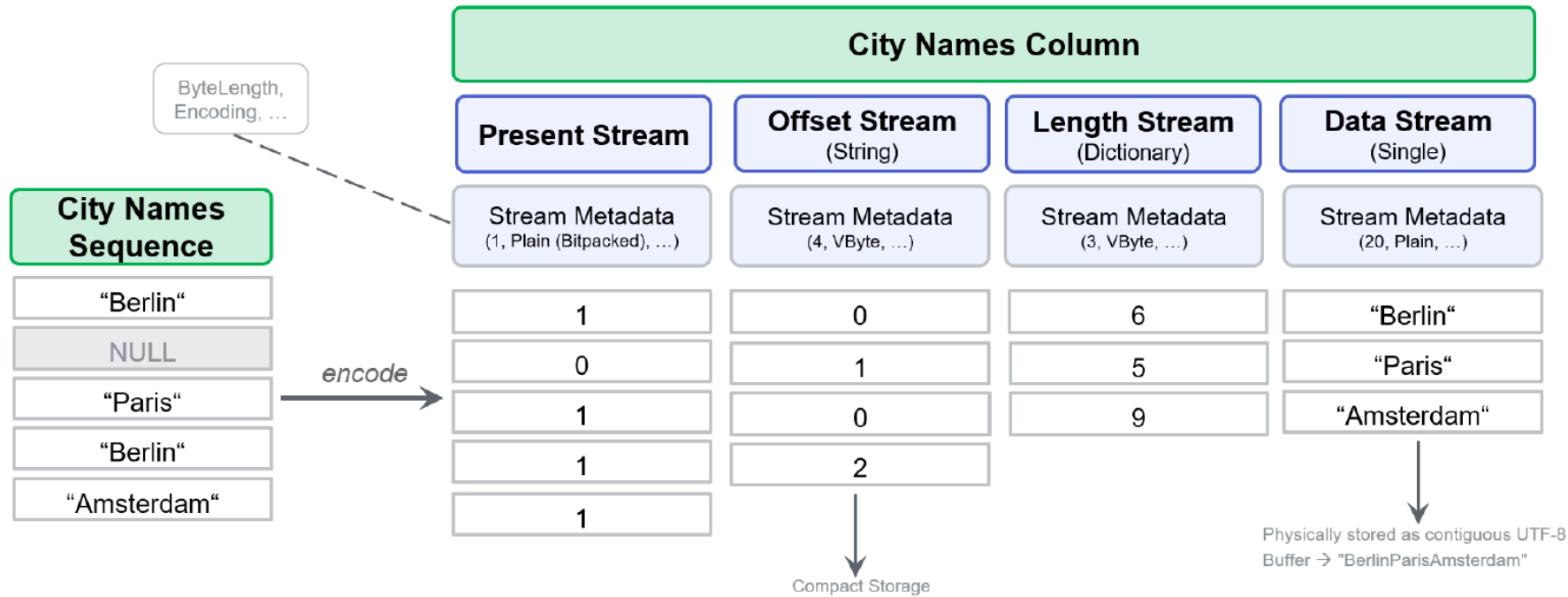
- the de facto industry standard for vector tile delivery, supporting up to 2.5D visualization
- stores feature data in a row-oriented layout
- relies heavily on general-purpose compression (gzip, Zstandard, ...)

► MapLibre Tile (MLT)

- the columnar successor designed for more efficient storage and decoding
- two coupled formats: a storage format and an in-memory format → Parquet ↔ Arrow, applied to tiles
- column-oriented layout combined with cascaded lightweight encodings
- pre-tessellated polygon meshes stored in the tile → skip tessellation on the client
- up to 3× smaller on uncompressed tiles and 2-3× faster to decode than MVT



MLT ATTRIBUTE ENCODING



MLT EXAMPLE

```
{ "type": "FeatureCollection",  
  "features": [  
    { "type": "Feature",  
      "id": 1,  
      "geometry": {  
        "type": "LineString",  
        "coordinates": [[12, 48], [14, 49]]  
      },  
      "properties": { "name": "A1", "pavementCondition": [4, 5] }  
    },  
    { "type": "Feature",  
      "id": 2,  
      "geometry": {  
        "type": "LineString",  
        "coordinates": [[11, 46], [12, 47]]  
      },  
      "properties": {  
        "name": "A2",  
        "pavementCondition": [8]  
        "speedLimit": [  
          {between: [0, 0.15], maxSpeed: 100},  
          {between: [0.15, 1], maxSpeed: 60}  
        ]  
      }  
    }  
  ]  
}
```

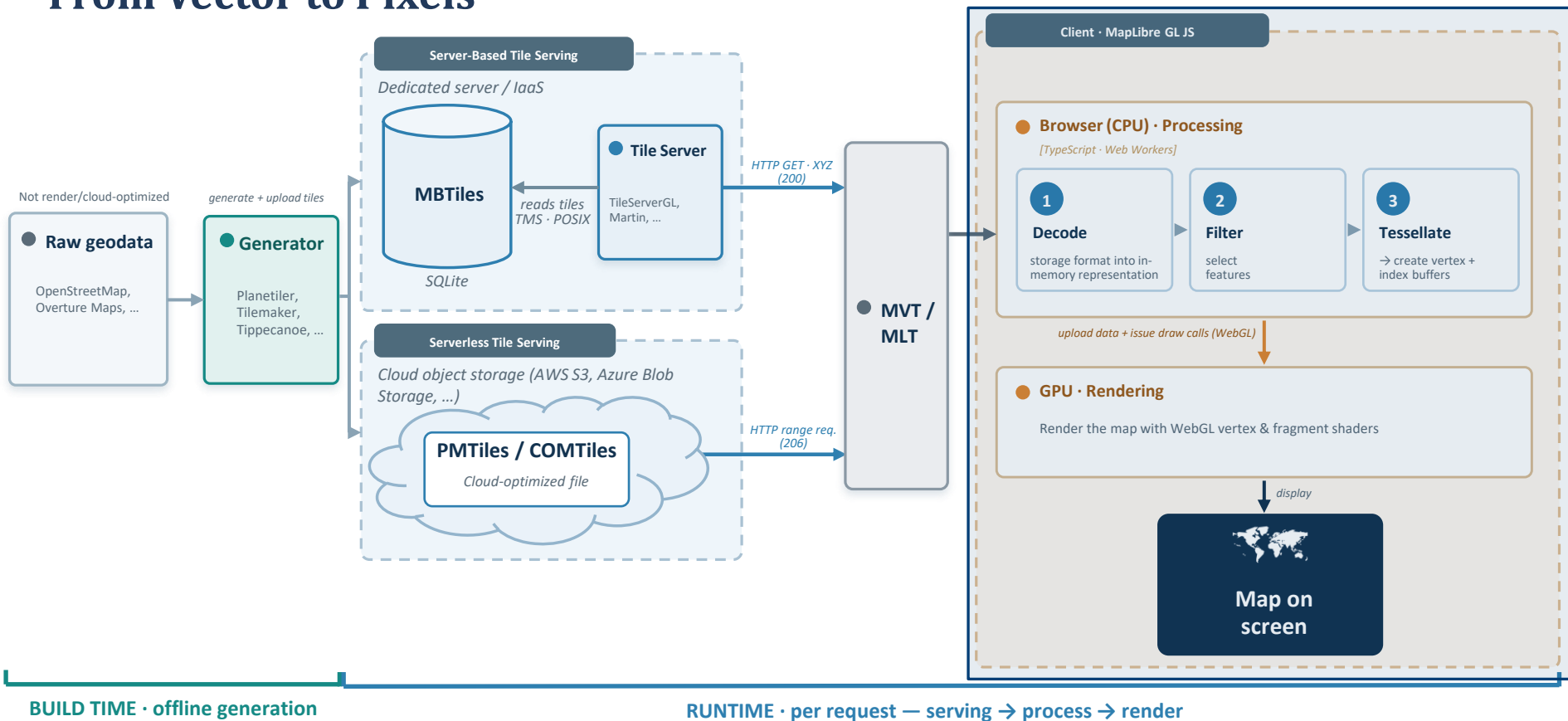
ID	Geometry		
Data	Geom Types	Num Vertices	Vertex Buffer
1	1	2	12
2	1	2	48
			14
			99
			...

Vertex-Scoped Attribute Columns	
pavementCondition	
Present	Data
true	4
true	5
true	8
false	

Feature-Scoped Attribute Columns	
name	
Length	Data
2	A
2	1
	A
	2

Geometry-Scoped Attribute Columns			
speedLimit			
Present	Length	Range	Data
false	2	0	100
true		0.15	60
		0.15	
		1	

From Vector to Pixels

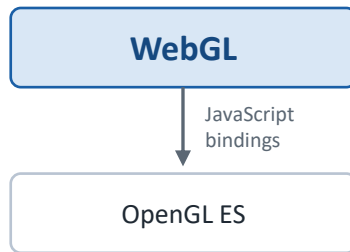
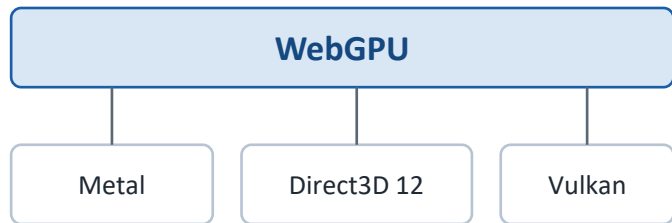


Graphics APIs for Rendering

Rendering turns raw vector data into an on-screen image. The browser offers four graphics APIs:

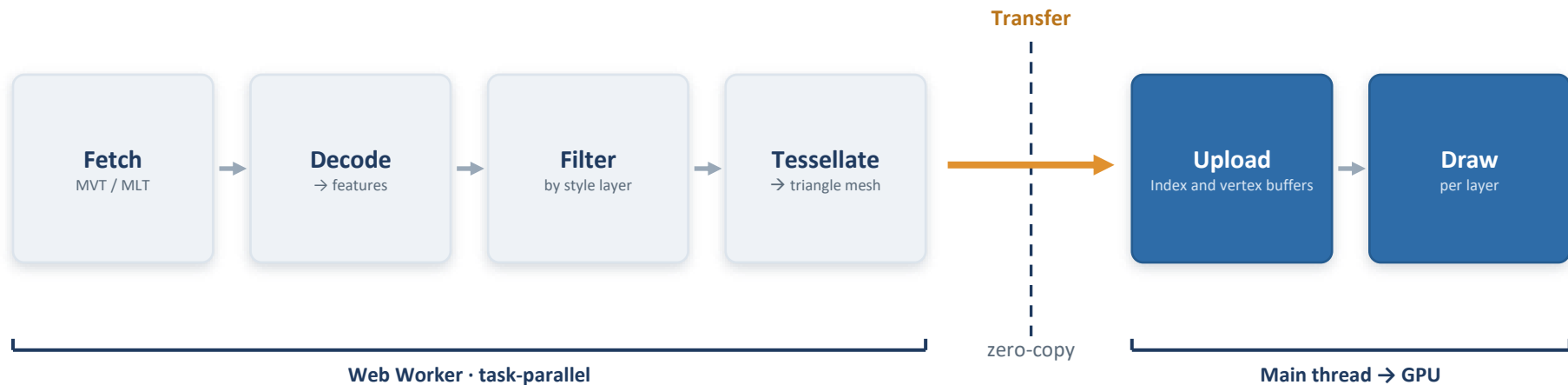


- WebGL brought the GPU into the browser → enabler for vector tile rendering in the browser
- WebGPU succeeds WebGL, exposing modern GPU APIs: Direct3D 12, Metal, Vulkan
- WebGPU adds compute shaders → GPGPU on the web, akin to CUDA / OpenCL

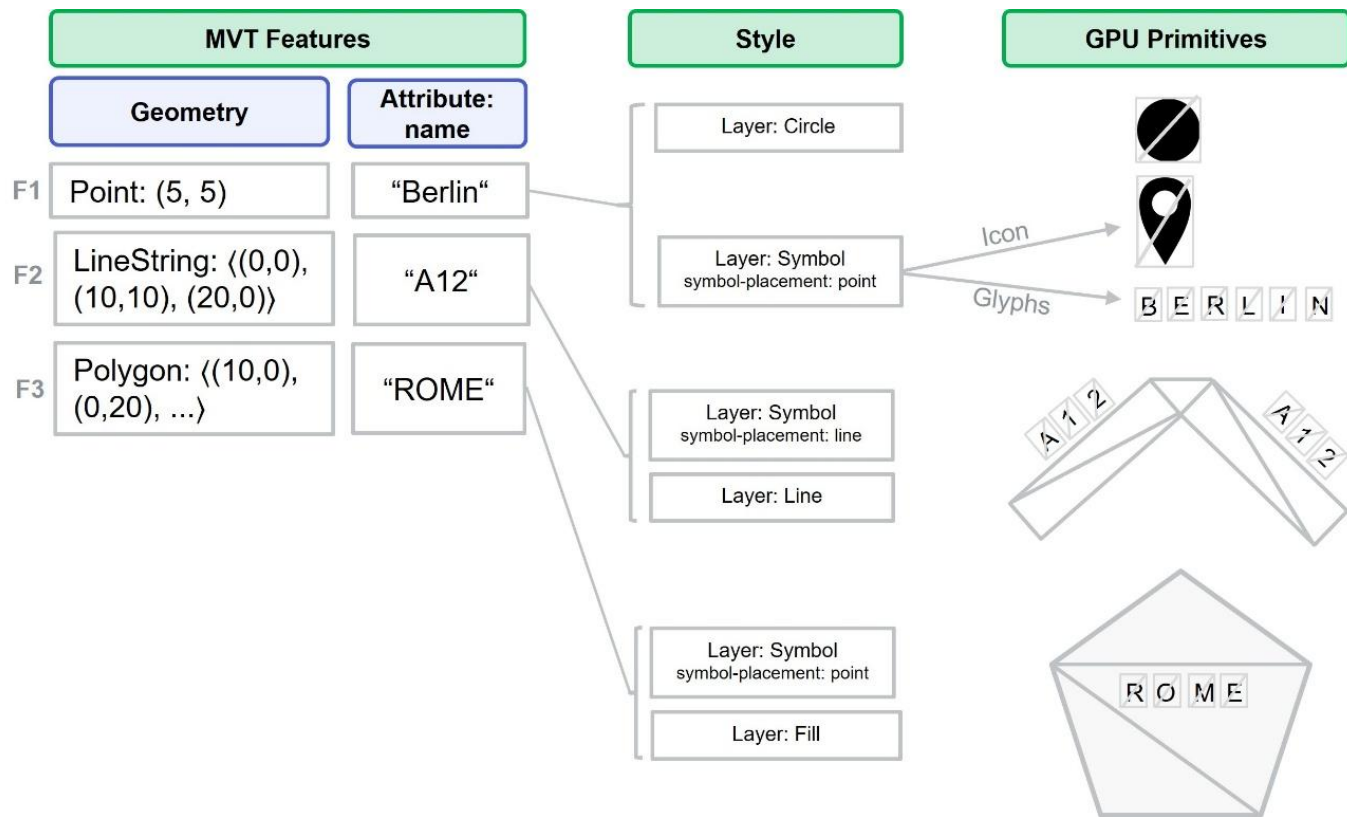


The full vector tile processing pipeline

- Anything longer than a frame must leave the main thread → otherwise the map stutters while you pan
- Workers hand back transferable ArrayBuffers → ownership moves, nothing is copied
- Task parallelism only → the heavy per-vertex work is still scalar JavaScript



FROM VECTOR DATA AND STYLE TO GPU PRIMITIVES



What's Next



OPTIMIZATION STRATEGIES VECTOR TILES RENDERING

State-of-the-Art Map Renderer Bottlenecks

Average US broadband speed 2025: ~ 267 Mbps

Throughput Line Tessellation: ~ 20 to 30 Mbps

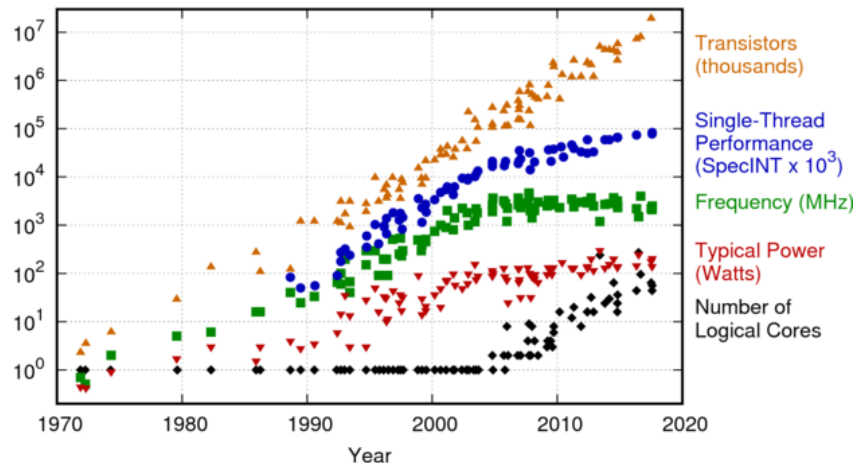
Throughput Decompress, Decode and Line
Tessellation: ~ 10 Mbps → broadband speed is
about factor 20 faster



Mapbox GL JS first release about 12 years ago

Average US internet speed 2014 → ~ 31 Mbps

42 Years of Microprocessor Trend Data



Original data up to the year 2010 collected and plotted by M. Horowitz, F. Labonte, O. Shacham, K. Olukotun, L. Hammond, and C. Batten
New plot and data collected for 2010-2017 by K. Rupp

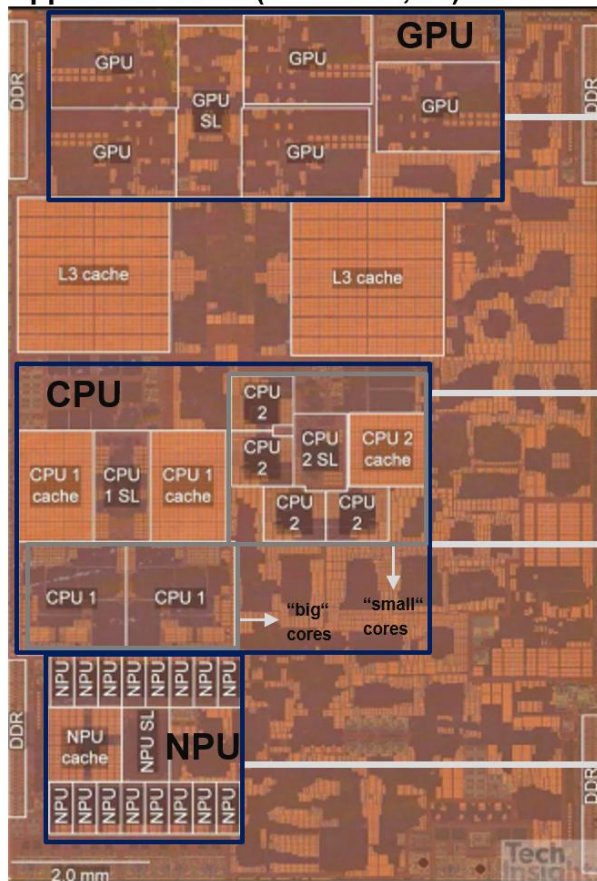
Source: [Karl Rupp](#)



**Parallelization → WebWorker, SIMD,
WebGPU Compute Shader**

HARDWARE TRENDS

Apple A15 Bionic (iPhone 13, 14)



→ No general-purpose compute (Graphics pipeline only)



→ Data Parallelism via Compute Shader

WebGPU

Web Worker → Task Parallelism



Web Assembly → Data Parallelism via SIMD



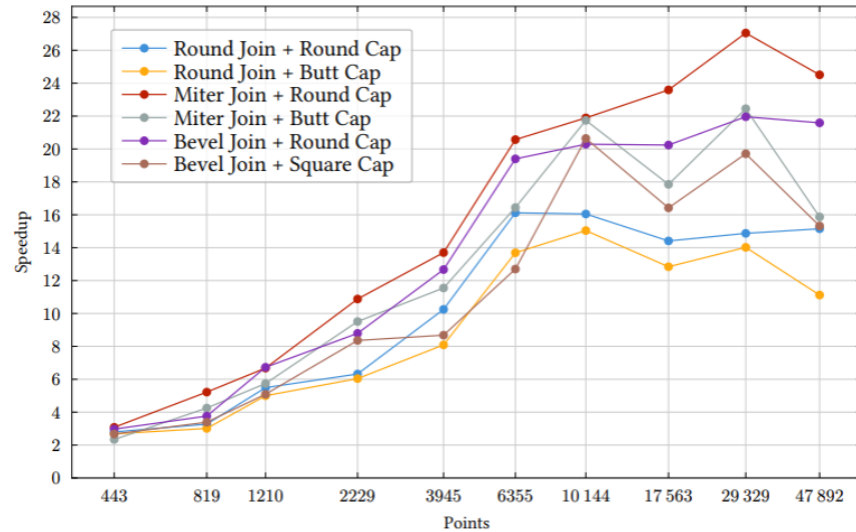
web
neural network

WebNN → Domain specific accelerator for DNNs (experimental)

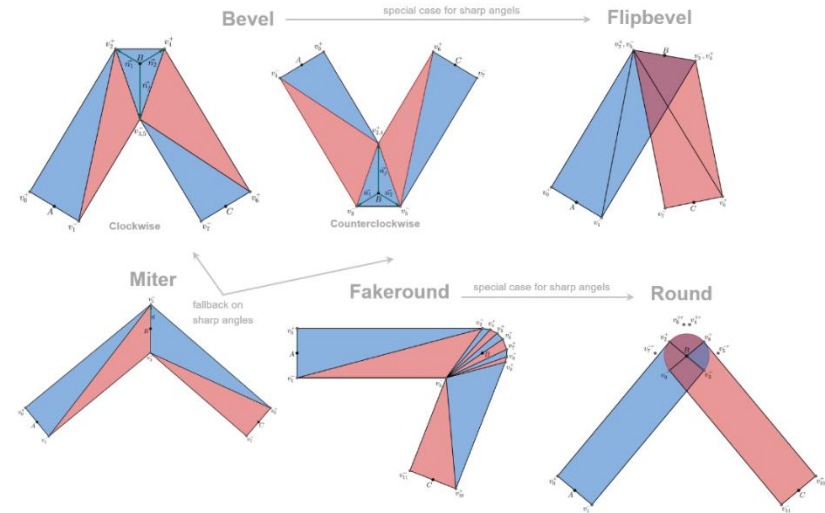
Source: Stanford CS149 (Parallel Computing) ·
Image credit: TechInsights Inc. ·
Web-API annotations added by author

GPU DRIVEN RENDERING

- Compute shaders enable GPU-driven rendering
- Example: compute-shader based line extrusion yields up to over an order-of-magnitude performance improvement



Source: Bachelor's thesis, Raphael Streule (supervised by author, Rohde & Schwarz)



IMMERSIVE 3D MAP VIEWS

**Cartography-Oriented 3D
Visualization (Symbolised 3D Models)**
Examples: Google Maps Immersive Navigation, Apple
Maps Detailed City Experience



Source: Apple Maps screenshot © Apple Inc.



MLT 3D

Photorealistic Rendering
(not a primary focus, as this is already well
supported by 3D Tiles)
Examples: Google Maps Photorealistic 3D Tiles



Source: Apple Maps screenshot © Apple Inc.

Screenshots: Apple Maps © Apple Inc.

FROM CLOUD TO GPU

THANK YOU QUESTIONS?

Markus Tremmel
FOSS4G Hiroshima 2026

ROHDE & SCHWARZ

Make ideas real

